CO5

ASSESSMENT TOOL-2

NAME : P. Ranjithkumar **DATE:19.08.2026**

REG.NO: 192411119 **Wednesday**

COURSE CODE : CSA1103

COURSE NAME : OOAD

Q1. Online Banking System

Question

Write pseudocode for a QA testing process that validates secure user authentication and fund transfer operations. Include input validation, transaction verification, exception handling, and test result logging.

Answer

The testing process verifies whether users can securely log in and perform fund transfers. It validates inputs before processing and records every test result.

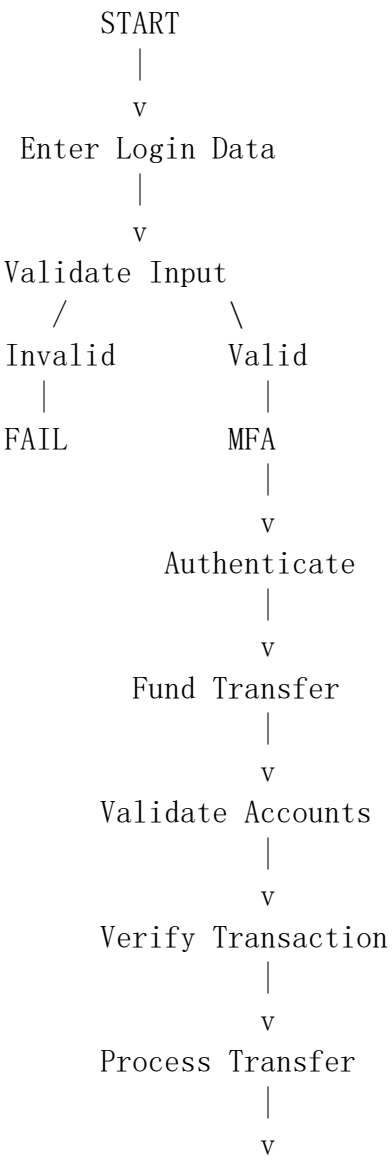
Pseudocode

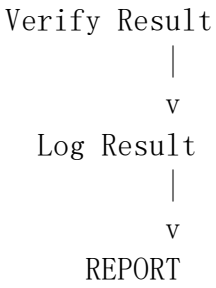
```
ALGORITHM OnlineBanking_QA_Test
    TRY

        IF senderAccount is empty OR receiverAccount is empty THEN
            LOG "Transfer Test", "FAIL", "Invalid account details"

        ELSE IF transferAmount <= 0 THEN
            LOG "Transfer Test", "FAIL", "Invalid amount"

        ELSE IF senderAccount = receiverAccount THEN
            LOG "Transfer Test", "FAIL", "Same account transfer"

        ELSE IF sender balance < transferAmount THEN
            LOG "Transfer Test", "FAIL", "Insufficient balance"

        ELSE
```

```
        CREATE transaction

        VERIFY sender account
        VERIFY receiver account
        VERIFY transaction amount

        IF all verification is successful THEN

            PROCESS transfer

            IF transfer is successful THEN

                VERIFY sender balance
                VERIFY receiver balance
                VERIFY transaction history

                IF all transaction records are correct THEN
                    LOG "Transfer Test", "PASS",
                        "Fund transfer successful"
                ELSE
                    LOG "Transfer Test", "FAIL",
                        "Transaction verification failed"
                END IF

            ELSE
                LOG "Transfer Test", "FAIL",
                    "Transfer processing failed"
            END IF

        ELSE
            LOG "Transfer Test", "FAIL",
                "Transaction verification failed"
        END IF

    END IF

CATCH AuthenticationException
    LOG "Transfer Test", "FAIL", "Authentication error"

CATCH DatabaseException
    LOG "Transfer Test", "FAIL", "Database error"

CATCH NetworkException
    LOG "Transfer Test", "FAIL", "Network error"

CATCH Exception
    LOG "Transfer Test", "FAIL", "Unknown error"
```

```
    END TRY

    ---------------------------------------------
    STEP 3: GENERATE TEST REPORT
    ---------------------------------------------

    CALCULATE total tests
    CALCULATE passed tests
    CALCULATE failed tests

    DISPLAY TestLog
    DISPLAY Test Summary

END
```

Diagram





Explanation

The algorithm first validates login credentials and MFA. After successful authentication, it validates sender and receiver accounts and the transfer amount. Transaction verification checks whether the correct amount has been deducted and credited. Exception handling prevents unexpected failures from terminating the test process. Finally, every test result is stored in a log containing **PASS/FAIL status and failure reason**.

Q2. E-Commerce Shopping Platform

Question

Design pseudocode for usability testing that measures product search time, checkout completion time, and navigation complexity.

Answer

The algorithm observes users while they perform shopping activities. It records timestamps, number of clicks, navigation steps, errors, and task completion status.

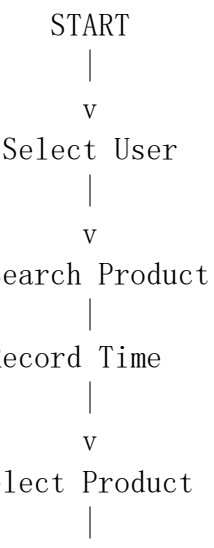
Pseudocode

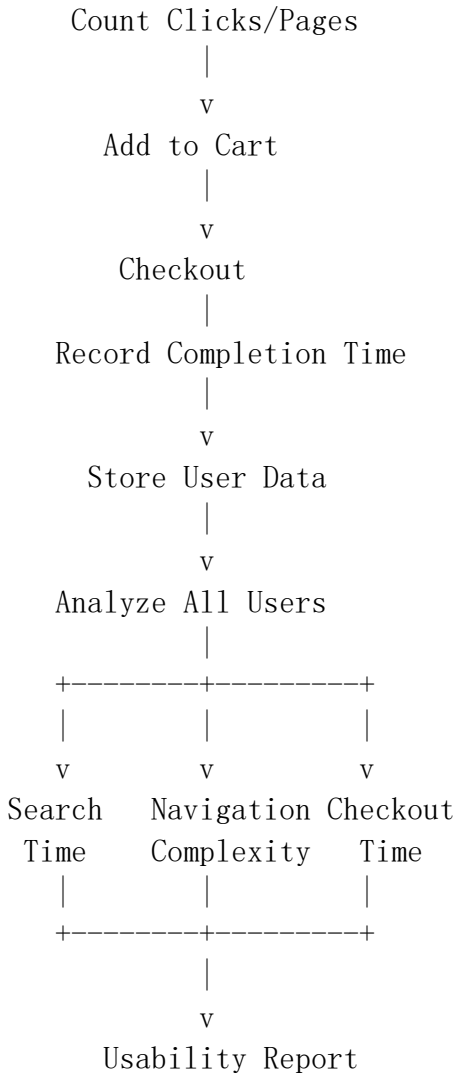
```
ALGORITHM ECommerce_Usability_Test

        START navigation counter

        USER selects product

        COUNT numberOfPagesVisited
        COUNT numberOfClicks

        navigationComplexity =
            numberOfPagesVisited + numberOfClicks

        LOG user,
            "Navigation",
            navigationComplexity
```

```
-----------------------------------------------
STEP 3: ADD TO CART
-----------------------------------------------

USER clicks "Add to Cart"

IF product added successfully THEN
    LOG user, "Cart", "PASS"
ELSE
    LOG user, "Cart", "FAIL"
END IF


-----------------------------------------------
STEP 4: CHECKOUT
-----------------------------------------------

checkoutStart = CURRENT_TIME

USER opens checkout

USER enters address
USER selects payment method
USER confirms order

checkoutEnd = CURRENT_TIME

checkoutTime =
    checkoutEnd - checkoutStart

COUNT checkoutSteps
COUNT checkoutErrors

IF order is successfully completed THEN
    checkoutStatus = "PASS"
ELSE
    checkoutStatus = "FAIL"
END IF

LOG user,
    "Checkout",
    checkoutTime,
    checkoutSteps,
    checkoutErrors,
    checkoutStatus
```

```
    END FOR

    ---------------------------------------------
    STEP 5: ANALYZE USER BEHAVIOR
    ---------------------------------------------

    CALCULATE averageSearchTime
    CALCULATE averageCheckoutTime
    CALCULATE averageNavigationComplexity
    CALCULATE taskCompletionRate
    CALCULATE averageErrors

    IF averageSearchTime is high THEN
        PRINT "Improve product search"
    END IF

    IF averageCheckoutTime is high THEN
        PRINT "Simplify checkout"
    END IF

    IF averageNavigationComplexity is high THEN
        PRINT "Improve navigation"
    END IF

    IF averageErrors is high THEN
        PRINT "Improve interface and validation"
    END IF

    GENERATE usabilityReport

END
```

Diagram





Data Collected

Metric	Purpose
Search Time	Measures how quickly users find products
Checkout Time	Measures purchase efficiency
Number of Clicks	Measures interaction effort
Pages Visited	Measures navigation complexity
Errors	Identifies usability problems
Task Completion	Measures overall success

Explanation

The algorithm records user behavior during each task. Timestamps are used to calculate search and checkout times. Clicks and visited pages indicate navigation complexity. Errors and incomplete tasks identify problematic areas. After collecting data from multiple users, averages and completion rates are calculated. If the values are high, the corresponding interface should be redesigned.

Q3. Hospital Management System

Question

Write pseudocode to generate and execute test cases for appointment scheduling. Include doctor availability, conflicts, patient accuracy, and invalid requests.

Answer

Appointment scheduling is critical because incorrect appointments can result in conflicts, delays, and poor hospital service. The algorithm generates different test cases and checks whether the system responds correctly.

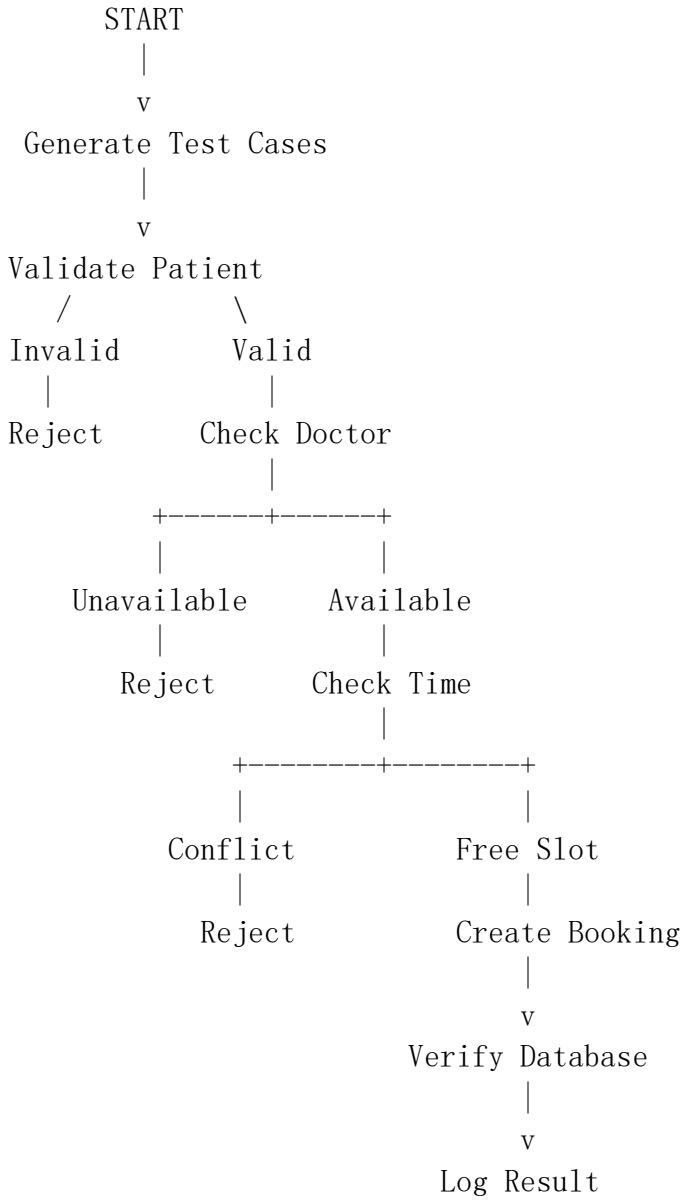
Pseudocode

```
ALGORITHM Hospital_Appointment_Test
                CONTINUE
            END IF


            ------------------------------------------------
            VALIDATE DOCTOR
            ------------------------------------------------

            IF doctor does not exist THEN
                LOG testCase, "FAIL",
                    "Invalid doctor"
                CONTINUE
            END IF

            IF doctor is unavailable THEN
                LOG testCase, "PASS",
                    "System rejected unavailable doctor"
                CONTINUE
            END IF


            ------------------------------------------------
            VALIDATE APPOINTMENT TIME
            ------------------------------------------------

            IF appointmentTime is invalid THEN
                LOG testCase, "PASS",
                    "Invalid time rejected"
```

```
        CONTINUE
    END IF

    IF appointmentTime is already booked THEN
        LOG testCase, "PASS",
            "Appointment conflict detected"
        CONTINUE
    END IF


    ------------------------------------------------
    CREATE APPOINTMENT
    ------------------------------------------------

    appointment = CREATE_APPOINTMENT(
                    patient,
                    doctor,
                    appointmentTime
                  )

    IF appointment created successfully THEN

        VERIFY appointment record

        IF record is correct THEN
            LOG testCase, "PASS",
                "Appointment created successfully"
        ELSE
            LOG testCase, "FAIL",
                "Incorrect appointment record"
        END IF

    ELSE
        LOG testCase, "FAIL",
            "Appointment creation failed"
    END IF


CATCH DatabaseException
    LOG testCase, "FAIL",
        "Database error"

CATCH Exception
    LOG testCase, "FAIL",
        "Unexpected system error"

END TRY
```

```
    END FOR

    ------------------------------------------------
    STEP 3: GENERATE REPORT
    ------------------------------------------------

    CALCULATE passedTests
    CALCULATE failedTests

    DISPLAY TestResults

END
```

Diagram



Important Test Cases

Test Case	Expected Result
Valid patient + available doctor	Appointment created
Invalid patient	Request rejected
Doctor unavailable	Request rejected
Already occupied slot	Conflict detected
Invalid date/time	Request rejected
Missing patient data	Validation error
Database failure	Exception handled

Explanation

The algorithm first creates normal, boundary, and invalid test cases. Each case validates patient information, doctor availability, and appointment time. If a conflict exists, the system must reject the request rather than creating a duplicate appointment. Exception handling manages unexpected database or system failures. Test results are stored so QA engineers can identify failed scenarios.

Q4. Airline Reservation System

Question

Develop pseudocode for constraint-based testing where booking requests must satisfy valid passenger information, seat availability, and payment confirmation.

Answer

In an airline reservation system, a booking is accepted only when **all constraints are satisfied**. If even one constraint fails, the booking should be rejected safely.

Constraints

```
C1 = Passenger information is valid
C2 = Flight exists
C3 = Seat is available
C4 = Payment is confirmed
C5 = Booking is not duplicated
```

The booking is valid only when:

```
VALID BOOKING =
```

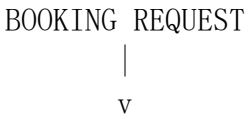
C1 AND C2 AND C3 AND C4 AND C5

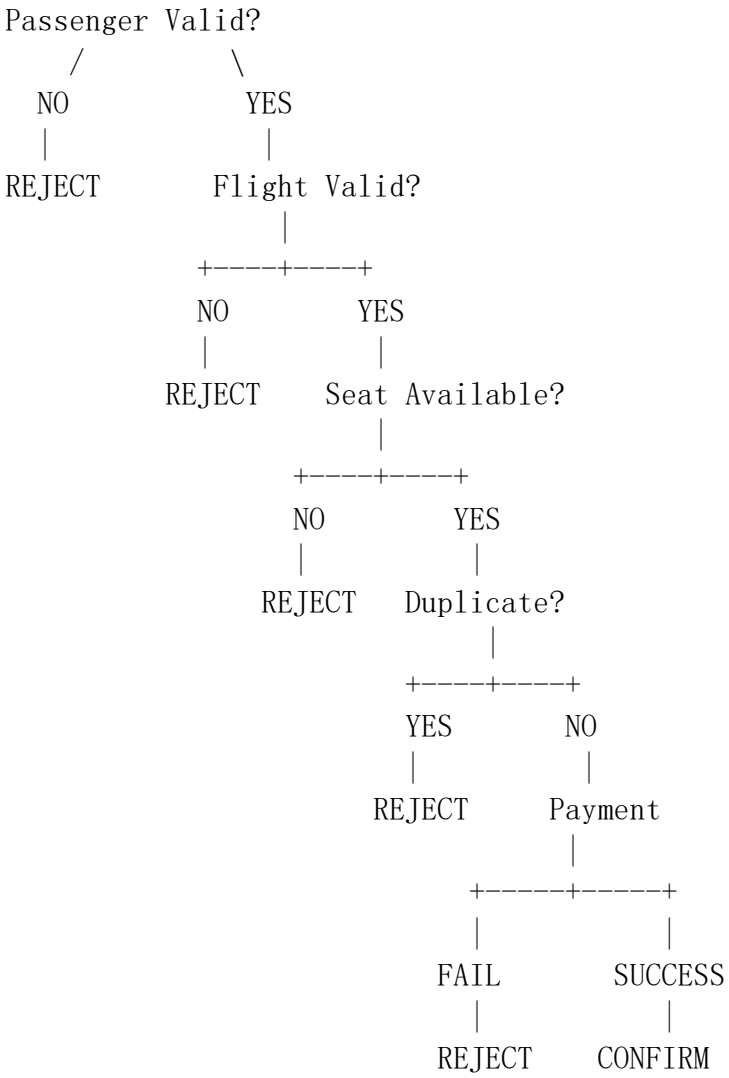
Pseudocode

```
ALGORITHM Airline_Constraint_Test
    IF flightID does not exist THEN
        PRINT "INVALID: Flight does not exist"
        LOG "FAIL - Invalid flight"
        STOP
    END IF


    ------------------------------------------------
    CONSTRAINT 3: SEAT AVAILABILITY
    ------------------------------------------------

    IF seatNumber is not available THEN
        PRINT "BOOKING REJECTED: Seat unavailable"
        LOG "FAIL - Seat constraint"
        STOP
    END IF


    ------------------------------------------------
    CONSTRAINT 4: DUPLICATE BOOKING
    ------------------------------------------------

    IF passenger already has same booking THEN
        PRINT "BOOKING REJECTED: Duplicate booking"
        LOG "FAIL - Duplicate booking"
        STOP
    END IF


    ------------------------------------------------
    CONSTRAINT 5: PAYMENT
    ------------------------------------------------

    TRY

        paymentStatus = PROCESS_PAYMENT(paymentDetails)

        IF paymentStatus != "CONFIRMED" THEN
            PRINT "BOOKING REJECTED: Payment failed"
            LOG "FAIL - Payment"
            STOP
        END IF
```

```
        ---------------------------------------------
        ALL CONSTRAINTS SATISFIED
        ---------------------------------------------

        LOCK seatNumber

        CREATE booking

        VERIFY booking record

        IF booking record is correct THEN
            CONFIRM booking
            GENERATE ticket

            LOG "PASS - Booking successful"

            PRINT "Booking Confirmed"
            PRINT "Ticket Generated"

        ELSE
            RELEASE seatNumber
            LOG "FAIL - Booking verification"
        END IF


    CATCH PaymentException

        PRINT "Payment service error"
        LOG "FAIL - Payment exception"

    CATCH DatabaseException

        PRINT "Database error"
        LOG "FAIL - Database exception"

    CATCH Exception

        PRINT "Unexpected error"
        LOG "FAIL - Unknown exception"

    END TRY

END
```

Constraint Diagram





Constraint Table

Constraint	Test	Invalid Response
Passenger information	Missing/incorrect data	Reject booking
Flight	Invalid flight ID	Reject booking
Seat	Already booked	Reject booking
Duplicate	Existing booking	Reject booking
Payment	Failed payment	Reject booking
All valid	All conditions satisfied	Confirm booking

Explanation

The algorithm follows a constraint-based approach. Each booking request is checked against a series of conditions. The booking is accepted only when passenger details, flight, seat, payment, and duplication checks are valid. If any constraint fails, the system rejects the request and records the reason. Seat locking prevents another user from booking the same seat during the transaction.

Q5. Smart Library Management System

Question

Write pseudocode for integrating QA and usability testing to evaluate functional correctness and user experience.

Answer

The Smart Library system must be tested from two perspectives:

Quality Assurance:
Checks whether book search, reservation, borrowing, and returning work correctly.

Usability Testing:
Checks response time, navigation complexity, accessibility, and ease of use.

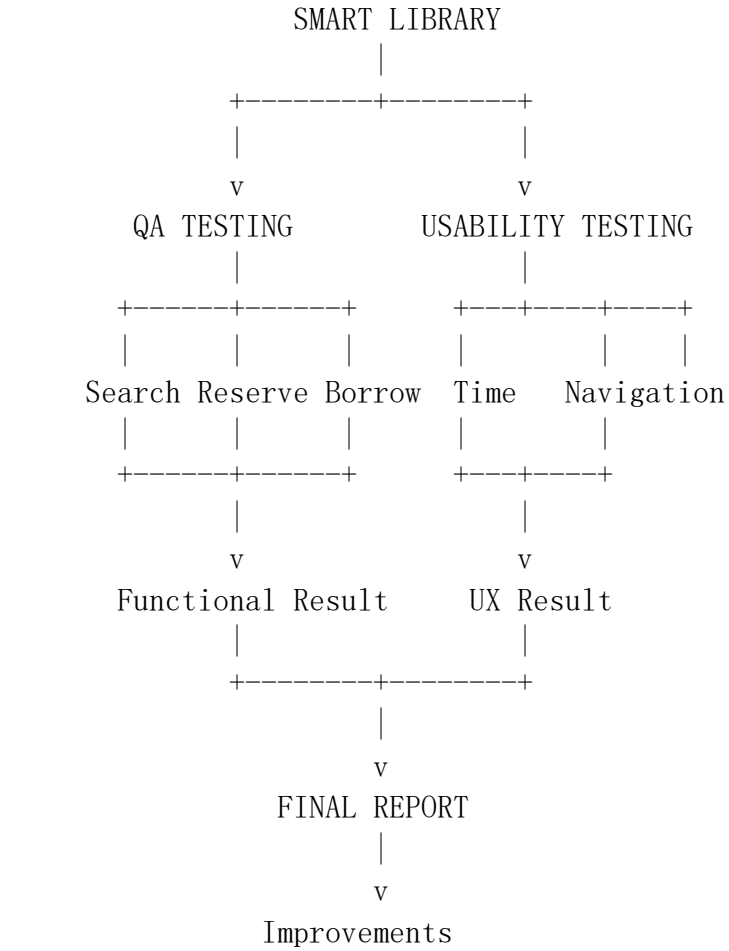
Pseudocode

```
ALGORITHM SmartLibrary_QA_Usability_Test
        COUNT searchClicks


        ------------------------------------------------
        NAVIGATION TEST
        ------------------------------------------------

        COUNT pagesVisited
        COUNT navigationClicks

        navigationComplexity =
            pagesVisited + navigationClicks


        ------------------------------------------------
        ACCESSIBILITY TEST
        ------------------------------------------------

        CHECK font readability
        CHECK color contrast
        CHECK button size
        CHECK keyboard navigation
        CHECK screen reader compatibility

        IF all accessibility checks pass THEN
```

```
        accessibilityStatus = "PASS"
    ELSE
        accessibilityStatus = "FAIL"
    END IF

    ---------------------------------------------
    USER TASK COMPLETION
    ---------------------------------------------

    IF user completes task successfully THEN
        taskStatus = "PASS"
    ELSE
        taskStatus = "FAIL"
    END IF


    LOG UsabilityTestLog,
        user,
        userSearchTime,
        searchClicks,
        navigationComplexity,
        accessibilityStatus,
        taskStatus

END FOR


-----------------------------------------------
PART 3: ANALYZE RESULTS
-----------------------------------------------

CALCULATE averageSearchTime
CALCULATE averageNavigationComplexity
CALCULATE taskCompletionRate
CALCULATE accessibilityPassRate

IF averageSearchTime > 2 seconds THEN
    PRINT "Improve search performance"
END IF

IF averageNavigationComplexity is high THEN
    PRINT "Simplify navigation"
END IF

IF accessibilityPassRate is low THEN
    PRINT "Improve accessibility"
```

```
    END IF

    IF taskCompletionRate is low THEN
        PRINT "Redesign user interface"
    END IF


    ---------------------------------------------
    FINAL REPORT
    ---------------------------------------------

    DISPLAY FunctionalTestLog
    DISPLAY UsabilityTestLog

    GENERATE FinalQAUsabilityReport

END
```

Combined QA + Usability Diagram



Functional Testing

The QA part checks whether:

- Search returns correct books.
- Reservation works correctly.
- Unavailable books cannot be reserved.
- Eligible users can borrow books.
- Ineligible users are rejected.
- Returned books become available.
- Database records are updated correctly.

Usability Testing

The usability part measures:

1. Response Time

```
Search Start → Search Results
```

The time should be measured and compared with the required limit.

2. Navigation Complexity

A simple measure can be:

```
Navigation Complexity =
Number of Pages Visited + Number of Clicks
```

A lower value generally indicates easier navigation.

3. Accessibility

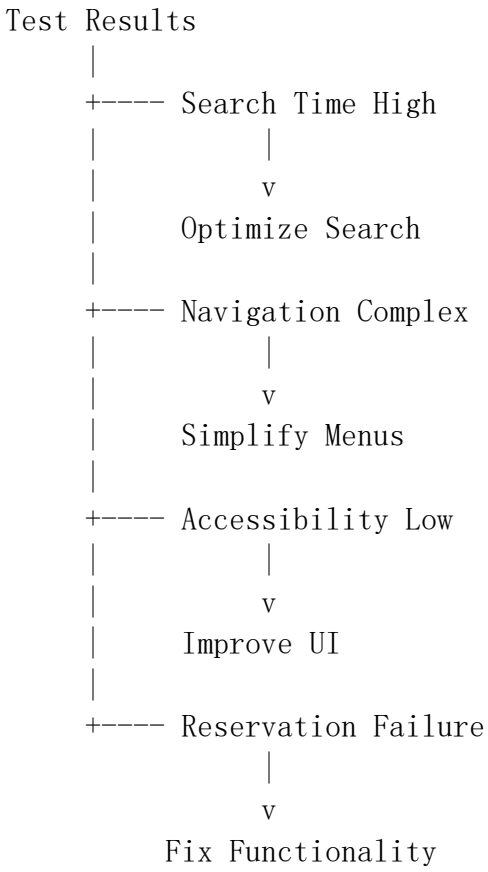
Check:

- Font readability.
- Color contrast.
- Keyboard accessibility.
- Screen-reader compatibility.
- Button size.
- Clear labels.

4. Task Completion

```
Task Completion Rate =
Successful Tasks / Total Tasks × 100
```

Example Analysis



Conclusion

The Smart Library system requires both functional QA and usability evaluation. Functional testing ensures that searching, reservation, borrowing, and returning operate correctly. Usability testing measures response time, navigation complexity, accessibility, and task completion. Combining both approaches helps identify not only whether the system **works correctly**, but also whether users can **use it easily and efficiently**. This results in a reliable, accessible, and user-friendly library management system.